

## E. 芽芽與 MSTP

### Description

網路的第二層架構負責的是區域網路 (Local Area Network, LAN)，例如每個學校、公司、家庭有各自的廣播網域 (Broadcast Domain)，每個 LAN 由多個交換器 (Switch) 和乙太網路線組成。第三層架構則是路由器 (Router)，連接著多個不同的 LAN。

我們很自然的把每個 LAN 裡的 switch 想成圖論中的節點，每條網路線類比成有權重 (傳輸成本) 的邊，那麼一個 LAN 就是一張連通的帶權無向圖。每個節點都可以發出廣播 (Broadcast)，訊框 (Frame) 會透過活躍的網路線傳到每個節點；但由於 switch 的特性，一旦活躍的邊形成了一個環 (Cycle)，frame 將在環上無限複製與循環，耗盡所有頻寬，稱為廣播風暴 (Broadcast Storm)。

為了避免 broadcast storm 並且降低總成本，芽芽想到了他曾在資訊之芽圖論課中學過的概念：最小生成樹。他的想法是：平時選出一棵最小生成樹  $T$ ，把在  $T$  上的邊都設定成活躍的，其餘則停用、進入備援。當有活躍的線路故障時，再重新計算，將備用邊調回活躍並加入生成樹。如此一來，活躍的邊在邏輯上永遠都是一棵生成樹，可以在線路沒有環的情況下，讓所有的節點連通，同時最小化總成本。於是，芽芽發明了這個改變第二層網路世界的協定：最小生成樹協定 (Minimum Spanning Tree Protocol, MSTP)。<sup>1</sup>

芽芽作為網管，已經掌握了這個 LAN 的每個 switch 和所有網路線的連接狀況，想立馬實作看看他的 MSTP。但在第一步就遭受了阻礙：他有選擇障礙，最小生成樹可能不只一種<sup>2</sup>，他不知道要挑哪一棵！苦惱了很久，他最後狠下心來，想請你幫他拔除盡量少的網路線，使剩下的圖仍然恰有 1 棵最小生成樹，這樣他就不用選擇了。請先向他報告，最少需要拿掉幾條網路線才能達成目標 (當然也可能不需要拔任何線)。<sup>3</sup>

### Input

輸入第一行有兩個以空白隔開的正整數  $N, M$ ，代表這個 LAN 有  $N$  個 switch 和  $M$  條已連接的網路線。接下來有  $M$  行，其中第  $i$  行有三個以空白隔開的正整數  $U_i, V_i, W_i$ ，代表第  $i$  條網路線連接著第  $U_i$  臺和第  $V_i$  臺 switch，且該線路的傳輸成本為  $W_i$ 。

- $2 \leq N \leq 2 \cdot 10^5$
- $1 \leq M \leq 2 \cdot 10^5$
- $1 \leq U_i, V_i \leq N$
- $1 \leq W_i \leq 10^9$
- 這是一張簡單圖： $U_i \neq V_i$ ，且任兩臺 switch 間最多只有一條網路線
- 這張圖是連通的：從任一臺 switch 發出廣播，可以透過這些網路線送到所有的 switch

<sup>1</sup>真正的 MSTP 是 Multiple Spanning Tree Protocol，而更為經典的 STP 則是維護類似最短路徑樹的結構。

<sup>2</sup>如果兩棵最小生成樹有任何一條邊不同，我們就說這兩棵最小生成樹不一樣。

<sup>3</sup>殊不知，芽芽就算要拔線也會面臨選擇障礙：要拔哪幾條？

**Output**

輸出一行一個非負整數，代表最少需要拿掉幾條網路線，才能讓剩下的圖有恰 1 棵最小生成樹。

**Sample 1**

| Input                                            | Output |
|--------------------------------------------------|--------|
| 4 5<br>3 1 4<br>2 4 9<br>4 3 4<br>3 2 9<br>2 1 9 | 2      |

**Sample 2**

| Input                                            | Output |
|--------------------------------------------------|--------|
| 4 5<br>3 1 4<br>2 4 9<br>4 3 4<br>3 2 4<br>2 1 9 | 0      |

**Sample 3**

| Input | Output |
|-------|--------|
|-------|--------|

|               |   |
|---------------|---|
| 7 20          | 3 |
| 2 3 4006328   |   |
| 5 4 183595960 |   |
| 6 5 183595960 |   |
| 2 5 183595960 |   |
| 3 1 506034667 |   |
| 7 2 4006328   |   |
| 2 1 506034667 |   |
| 2 4 516911770 |   |
| 1 6 4006328   |   |
| 4 1 516911770 |   |
| 5 7 183595960 |   |
| 3 6 506034667 |   |
| 3 7 183595960 |   |
| 7 1 183595960 |   |
| 3 5 506034667 |   |
| 4 3 516911770 |   |
| 7 6 516911770 |   |
| 2 6 506034667 |   |
| 4 6 4006328   |   |
| 7 4 516911770 |   |

## 配分

在一個子任務的「測試資料範圍」的敘述中，如果存在沒有提到範圍的變數，則此變數的範圍為 Input 所描述的範圍。

| 子任務編號 | 子任務配分 | 測試資料範圍          |
|-------|-------|-----------------|
| 1     | 0%    | 範例測試資料          |
| 2     | 10%   | 所有線路的傳輸成本都相同    |
| 3     | 30%   | 所有線路只有恰 2 種傳輸成本 |
| 4     | 60%   | 無特別限制           |

對於每一個子任務，若你於所有測試資料都有正確判斷答案是不是 0：成功在答案是 0 時輸出 0、答案不是 0 時輸出一個 1 到  $10^9$  間的整數，你就可以獲得該子任務 50% 的分數。

## Hint 1

本題測試資料量大，建議使用 `scanf` 進行輸入。若使用 `std::cin` 輸入，請在 `main` 函式第一行加上 `ios_base::sync_with_stdio(0); cin.tie(0);`，且請勿跟 `scanf` 混用，以

免造成 Time Limit Exceeded。

## Hint 2

不要因為這題放在最後一題就不來做！聽說考試時可以看課程網的上課簡報什麼的，想想課堂上教的證明邏輯是什麼，也可以搭配子任務服用一下。加油～